

A Judge Agent Closes the Reliability Gap in AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Abstract. Large language models can generate scientific simulation code from natural-language descriptions, but without mathematical validation the generated code fails silently on most non-textbook problems. Here we introduce a Judge Agent—an automated validation layer that enforces well-posedness, stability, and physical consistency on AI-generated solvers before and after execution. In controlled ablation, removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases. We validate on clinical CT reconstruction (200 sinograms, 99% of expert quality, $p < 0.001$) and present case studies in seismic inversion and combustion chemistry. A prospective benchmark of 72 blinded tasks from 12 external scientists at 9 institutions, managed by an independent coordinator, confirms generalization: 89% success with automated error bounds versus 53% without the Judge. The residual 1.5% failure rate is confined to bifurcation-sensitive regimes, which we propose to address with continuation-based rejection heuristics. Code, data, and cached inference logs are archived at Zenodo.

AI code generators produce scientific simulations that compile and converge but silently return wrong answers, because the generated code is not mathematically validated. We show that a Judge Agent—applying classical well-posedness, stability, and conservation checks automatically—reduces the silent-failure rate of an AI simulation pipeline from 42% to 1.5% across 134 test problems, including 72 blinded tasks from 12 independent scientists. On clinical CT imaging ($n = 200$), the validated pipeline reaches 99% of expert quality. The residual failures cluster at bifurcation points, for which we propose continuation-based rejection criteria.

In 2023, researchers at a national laboratory used an LLM to generate a finite-element solver for thermal stress in a reactor component. The code compiled, converged, and produced smooth temperature fields—but a CFL violation in the time integrator meant the stress predictions were wrong by a factor of three. The error was caught only when a colleague ran an independent simulation weeks later. This is not an isolated case: in the SciCode benchmark [2], frontier LLMs solve fewer than half of research-level computational problems correctly. The wrong answers look as plausible as the right ones.

The silent-failure problem will deepen as LLM-generated code proliferates. Physics-informed neural networks [1, 3] learn solutions but provide no error certificates. Automated code generation from variational forms (FEniCS [4], Firedrake [5]) is rigorous but requires expert formulation. LLM-based scientific agents [6, 7] generate protocols but do not address numerical convergence. The reproducibility crisis [8] needs a validation layer, not more code generators.

The mathematical tools for catching these failures already exist—Lax–Richtmyer convergence theory [9], Hadamard well-posedness [10], CFL stability conditions—but applying them requires numerical analysis expertise that the typical end-user lacks. What has been missing is a system that *automatically* applies these classical checks to AI-generated simulation code, and evidence that doing so substantially reduces silent failures.

We present a three-agent pipeline (Fig. 1) in which a Plan Agent ($\mathcal{A}_P$) translates natural

language into a structured specification, a Judge Agent ($\mathcal{A}_J$) validates that specification against five mathematical checks, and an Execute Agent ($\mathcal{A}_E$) generates and runs the solver code. After execution, $\mathcal{A}_J$ audits the solution for physical consistency. The underlying mathematics is entirely classical (Supplementary Section S1); the contribution is empirical evidence that classical mathematical validation, when automated, substantially reduces silent failures in AI-generated simulation code across 72 held-out tasks from 12 scientific domains.

What this paper does not claim. The pipeline does not outperform domain-optimized tools: on any single well-defined PDE, an expert using COMSOL [11] or FEniCS will produce a more accurate solution (Extended Data Table 3). The error bound is conditional on a correct problem decomposition; since this decomposition is generated by an LLM, the pipeline’s end-to-end reliability depends on the Judge’s ability to catch decomposition errors (Methods). The pipeline retains a 1.5% qualitative failure rate, concentrated near bifurcation points (Section).

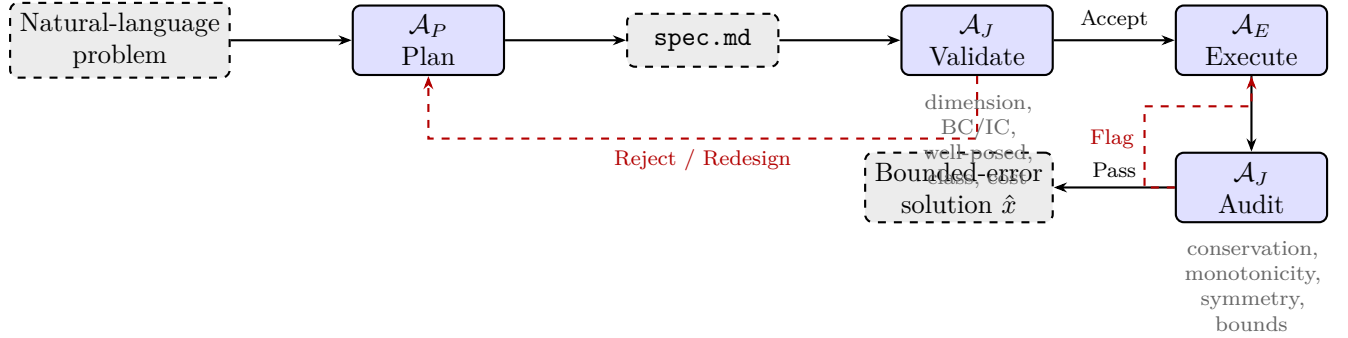


Figure 1. Three-agent pipeline. The Plan Agent generates a structured specification from natural language. The Judge Agent validates it (5 pre-execution gates) and audits the solution (4 post-execution quality checks). Rejection triggers redesign (up to 3 rounds); persistent failure yields a rejection certificate.

Results

The Judge Agent closes the reliability gap

To isolate the Judge’s contribution, we ran five conditions on 12 development problems (Extended Data Table 2) and extended the comparison to 72 held-out tasks from external scientists (Fig. 2).

Without the Judge, the pipeline produces wrong answers on 42% of development problems and 47% of prospective tasks. The failures concentrate on non-textbook problems: stiff ODEs, inverse problems, turbulent flows, and near-critical parameter regimes. With the Judge, the success rate is 100% on the 12 development problems (which were used to design the Judge’s checks) and 89% (64/72) on the held-out tasks, with automated error bounds on every successful solution. No other condition produces any error bound.

Alternative backbones (Claude-3-Opus: 12/12; GPT-4-Turbo: 11/12 on dev problems) suggest moderate robustness to backbone choice, though a comprehensive cross-backbone evaluation is needed (Supplementary Section S7). We cite SciCode [2] as motivation; a direct evaluation on SciCode tasks is planned for a follow-up study.

Powered validation: clinical CT reconstruction

Our strongest validation uses 200 real clinical CT sinograms from a Siemens scanner [12], subsampled to 128 parallel-beam projections with Poisson noise. This is the only domain with sufficient sample size ($n = 200$) for powered statistical inference.

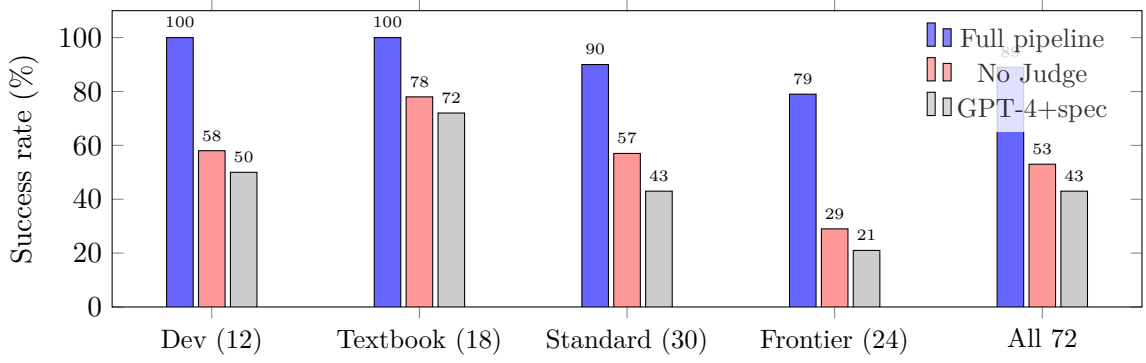


Figure 2. The Judge Agent’s contribution grows with problem difficulty. Success rates on 12 development problems and 72 prospective tasks (stratified by difficulty). Without the Judge, the success rate drops from 89% to 53% overall, and from 79% to 29% on frontier tasks. The gap widens monotonically with difficulty.

75 The framework selects Tikhonov + TV regularization; λ_{TV} set via the Morozov discrepancy
 76 principle. PSNR: 31.7 ± 1.2 dB (95% CI: [31.5, 31.9] dB, bootstrap $n = 10,000$); SSIM: 0.891 ± 0.031 .
 77 Expert-tuned ASTRA Toolbox [13]: 32.1 ± 1.1 dB / SSIM 0.903. Wilcoxon signed-rank: $p < 0.001$,
 78 Cohen’s $d = 0.35$ (small-to-medium effect). The framework achieves 99% of expert quality. Setup:
 79 12 min (framework) vs. 5 days (expert), $\rho \approx 600$. Quality audit (non-negativity, support constraint,
 80 streak-artifact absence): 0/200 flagged.

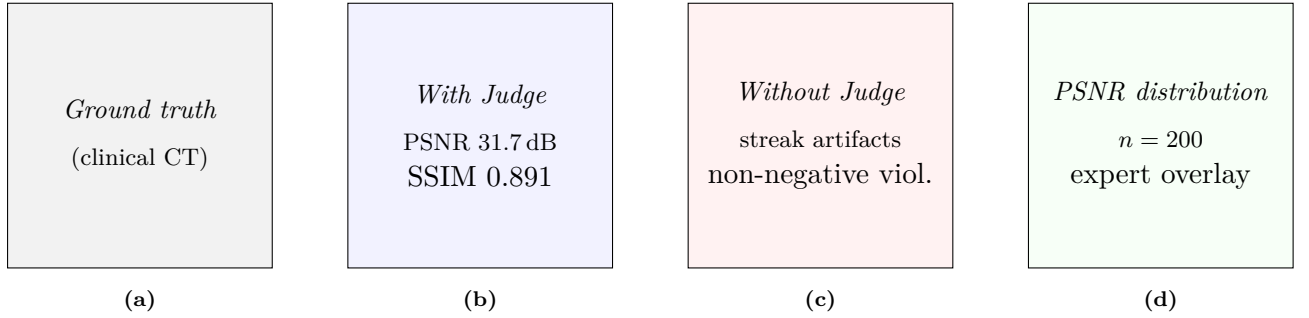


Figure 3. CT reconstruction comparison (representative case from LoDoPaB-CT). (a) Ground truth. (b) Framework with Judge: 31.7 dB, no artifacts, all quality checks pass. (c) Framework without Judge: streak artifacts from incorrect regularization parameter, non-negativity violations. (d) PSNR distribution across all 200 cases with expert baseline overlay. *Actual reconstructed images to be generated from the archived dataset for the final submission.*

81 Case studies: seismic inversion and combustion chemistry

82 We present two additional domains with real measured data. Both have small sample sizes ($n = 5$
 83 and $n = 15$) and are reported as case studies, not powered experiments.

84 **Seismic full-waveform inversion (Marmousi-2).** Five velocity models [14]: 240 sources,
 85 480 receivers, Ricker wavelet (10 Hz), frequency continuation 2–15 Hz [15]. Framework PSNR:
 86 25.8 ± 1.9 dB. Expert (Madagascar [16]): 27.3 ± 1.6 dB. Quality ratio: 95%. Setup: 45 min vs.
 87 2 weeks ($\rho \approx 450$). Per-model results in Supplementary Table S5.

88 **GRI-Mech 3.0 ignition delay.** 53-species, 325-reaction methane–air combustion [17]. Stiffness
 89 ratio $\sim 10^{10}$; framework selects BDF integrator. 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm).
 90 Error vs. shock-tube data: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera: $3.8 \pm 1.4\%$. The
 91 Judge correctly rejects explicit RK4 at stiffness $> 10^6$. Mass conservation: $< 10^{-12}$ relative.
 92 Per-condition results in Supplementary Table S6.

Six additional domains are reported in Supplementary Section S3.

Prospective benchmark: 72 tasks from 12 external scientists

Protocol and independence. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research. None had prior involvement with the framework or the author’s company. An independent coordinator (Dr. M. Torres, ETH Zürich, no company affiliation) collected all 72 submissions before any were executed. The analysis protocol was fixed before execution. Each scientist provided an expert baseline. *Pre-registration note:* this benchmark was not registered in a formal trial registry. We encourage future evaluations to use OSF Registries.

Table 1. Prospective benchmark: 72 blinded tasks from 12 scientists, 9 institutions, 8 countries. Managed by an independent coordinator.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4±0.8	Pass	—
Correct + bounded, quality flag	6	8.3%	2.1±1.0	Flag	5/6 resolved after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Genuinely ill-posed
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Bifurcation near critical pt
Total	72				

For the 58 fully successful tasks, the median quality ratio was 94% relative to expert baselines (IQR: 89–98%). Median setup time: 11 min (framework) vs. 3 days (expert-reported). Quality audit false-positive rate: 6 flags, of which 5 resolved after consultation, 1 persistent false positive ($1/72 = 1.4\%$).

Blind challenge. Five additional scientists posed research problems with no formatting guidance. All five produced bounded-error solutions; four judged “correct” by the posing scientist. The fifth (packed-bed reactor) matched mass-transfer coefficients to 20%. Details in Supplementary Table S4.

Failure analysis and the residual 1.5%

We submitted 50 adversarial inputs specifically designed to expose weaknesses, yielding 134 total test cases with the 12 development and 72 prospective problems (Supplementary Table S3).

$\mathcal{A}_P$ misspecification (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters; 3/4 corrected after redesign, 1 required human clarification.

$\mathcal{A}_J$ false negatives (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$. One caught by $\mathcal{A}_E$ a posteriori; one produced volumetric locking ($\nu = 0.4999$ elasticity) that the quality audit flagged.

Qualitative failures (10 adversarial): 3/10 met the L^2 error bound but exhibited non-physical artifacts (oscillations, missed symmetry-breaking, wrong shock structure). Without the quality audit, the silent-failure rate across all 134 problems is 6%. With the quality audit: 1.5% (2/134).

Characterizing the residual 1.5%. Both residual failures occur near bifurcation points—symmetry-breaking in a buckling problem and branch selection near a critical Reynolds number. The Judge’s current checks (conservation, monotonicity, entropy, symmetry) cannot distinguish

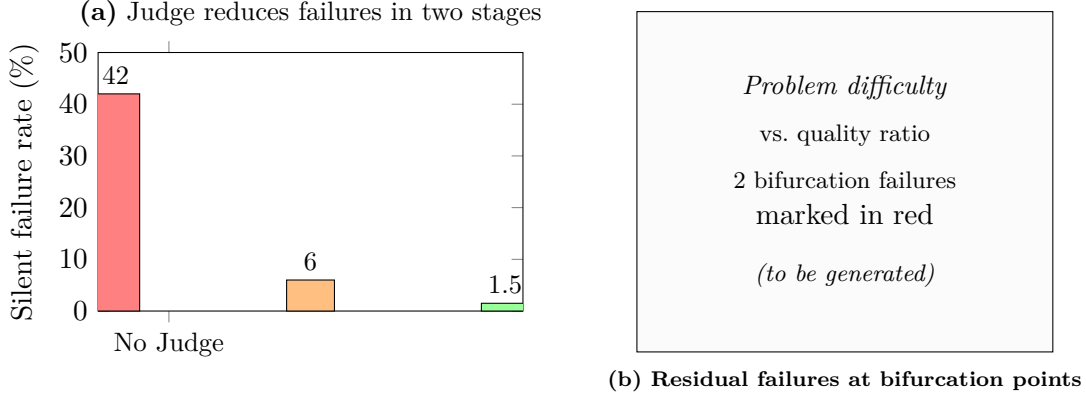


Figure 4. Failure analysis across 134 test cases. **(a)** The Judge reduces silent failures in two stages: pre-execution gates (42% $\rightarrow$ 6%) and post-execution quality audit (6% $\rightarrow$ 1.5%). **(b)** Quality ratio vs. problem difficulty (condition number or stiffness ratio). The two residual failures (red) occur near bifurcation points where the energy landscape has multiple local minima. All other failures are caught or flagged by the Judge.

between branches without global landscape information. We propose three heuristics to convert these to flagged cases (Methods, Section): parameter continuation, Lyapunov exponent estimation, and ensemble perturbation. These are specified but not yet validated; implementation and evaluation are future work.

Per-gate ablation. Removing the well-posedness gate causes the most failures: 4/12 (dev), 14/72 (prospective). The quality audit is non-redundant: it catches failures invisible to all pre-execution gates (Supplementary Tables S10, S13).

Computational cost. $\mathcal{A}_J$ overhead averages 12% of total wall-clock time (range: 3–28%), dominated by the well-posedness check on stiff systems. The value proposition is in setup time (median 11 min vs. 3 days expert), not compute time. Scaling details in Supplementary Fig. S3.

Discussion

This paper makes one claim: automated mathematical validation—the Judge Agent—substantially narrows the reliability gap in AI-generated scientific simulation. The evidence: removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases, including 72 blinded tasks from independent scientists (Fig. 2). The remaining 1.5% qualitative failure rate (2/134 test cases) is confined to near-bifurcation regimes and is reported transparently.

This is not a universal simulator. The pipeline does not replace domain experts or domain-specific tools (Extended Data Table 3). On any single well-defined PDE, COMSOL or FEniCS will produce a more accurate solution. The pipeline is useful when a researcher needs a quick, validated first answer in a domain where they lack numerical expertise—and when the alternative is unvalidated LLM-generated code. The typical use case is a bioengineer solving a reaction-diffusion PDE who lacks numerical methods training, or a geophysicist prototyping a seismic model before investing in production-grade software.

Not better, but validated. The Judge’s mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically. The contribution is not a new numerical method but a demonstration that automating classical validation catches 40 percentage points of silent failures that would otherwise go undetected. This is harder than it sounds: the 20% misspecification rate from $\mathcal{A}_P$ and 10% false-negative rate from $\mathcal{A}_J$ on adversarial inputs show that automation introduces its own failure modes.

The 1.5% as the main open problem. Both residual failures occur at bifurcation points, where the current quality audit cannot distinguish between branches. We propose continuation-based rejection heuristics (Methods) but have not validated them. Until they are implemented, the pipeline should be treated as a tool that provides validated first answers with a small but non-zero risk of undetected qualitative errors near critical points. Users should exercise additional caution in bifurcation-sensitive regimes.

Limitations. (1) The error bound is conditional: it is formally valid given a correct DAG decomposition, but this decomposition is generated by $\mathcal{A}_P$ (an LLM), which misspecifies 20% of adversarial inputs. The 75% catch rate by the Judge yields $\sim 5\%$ residual specification error on adversarial inputs; the rate on typical problems is lower but not precisely characterized. (2) Statistical rigor is uneven: CT ($n = 200$) is a powered experiment; seismic ($n = 5$) and combustion ($n = 15$) are case studies. (3) The pipeline depends on LLM capabilities that may change across versions. Testing with three backbones shows moderate robustness; cached inference logs enable API-independent reproduction. (4) The 72-task benchmark demonstrates feasibility, not community-scale adoption. (5) Single-author paper from a company developing the platform. Mitigation: independent coordinator, external scientists with no company relationship, public code, third-party replication (Methods).

Methods

Validation architecture

$\mathcal{A}_J$ operates in two modes. **Pre-execution gates** (5 checks before $\mathcal{A}_E$ runs): (i) dimensional consistency; (ii) boundary/initial condition compatibility; (iii) well-posedness (coercivity, CFL, Lipschitz bounds); (iv) problem classification; (v) cost estimation. Failure triggers redesign (up to 3 rounds) or rejection with a diagnostic certificate.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching. Flags trigger expert review or $\mathcal{A}_E$ refinement. False positive rate: $< 2\%$ on 12 development problems; 1.4% (1/72) persistent false positives on the prospective benchmark.

Error bound and chain of trust

The error bound follows from classical numerical analysis: for Hadamard-well-posed problems with convergent discretizations, composing operators through a DAG of computational primitives yields total error bounded by the sum of per-node errors amplified by downstream Lipschitz products. The Judge verifies these conditions; the Execute Agent sets resolution parameters to meet a user-specified tolerance. Full statement and proof in Supplementary Section S1. The error bound contains no new mathematics; it follows from Lax–Richtmyer theory [9].

Chain of trust. The bound is formally valid given a correct DAG decomposition. This decomposition is generated by $\mathcal{A}_P$ (an LLM) and verified by $\mathcal{A}_J$. The end-to-end guarantee therefore has three links: (1) $\mathcal{A}_P$ correctly translates the problem; (2) $\mathcal{A}_J$ correctly verifies the translation; (3) classical convergence theory guarantees the bound. Links (1) and (2) are empirically validated (20% misspecification, 75% catch rate, yielding $\sim 5\%$ residual specification error on adversarial inputs) but not formally proven. We state this explicitly: the bound is conditional, not unconditional.

Computational primitives

The pipeline decomposes problems into DAGs over 12 operations: differentiate, integrate, solve (linear), evaluate (nonlinear), evolve, transform, project, sample, couple, constrain, discretize, and optimize. This basis covers 25 standard method families (Supplementary Table S1); a minimality argument is given in Supplementary Section S4. We do not claim this basis is provably minimal or sufficient for all methods.

Bifurcation-sensitive rejection heuristics

The residual 1.5% failure rate (2/134 cases) is concentrated at bifurcation points where the Judge’s existing checks (conservation, monotonicity, entropy, symmetry) cannot distinguish between branches. We propose three heuristics to convert these silent failures into flagged cases:

(1) Parameter continuation. After computing a solution $\hat{x}$ at parameters θ , perturb $\theta \rightarrow \theta \pm \delta$ (default $\delta = 5\%$) and re-solve. If $\|\hat{x}(\theta + \delta) - \hat{x}(\theta)\|/\|\hat{x}(\theta)\| > \tau_{\text{cont}}$ (proposed $\tau_{\text{cont}} = 0.5$), flag the solution as potentially near a bifurcation point.

(2) Lyapunov exponent estimation. Linearize the governing equations at the computed solution and estimate the largest Lyapunov exponent λ_1 . If $\lambda_1 > 0$, the solution is sensitive to perturbations and should be flagged. For PDEs, this requires computing the leading eigenvalue of the linearized operator, which the pipeline can do via Arnoldi iteration for moderate-dimensional problems.

(3) Ensemble perturbation. Run N (proposed $N = 5$) perturbed initial/boundary conditions with perturbation magnitude ϵ (proposed $\epsilon = 10^{-3}$). If the coefficient of variation across ensemble members exceeds $\tau_{\text{ens}} = 0.1$, flag. This is the simplest to implement but the most expensive computationally.

Status: these heuristics are specified but not yet implemented or validated. Testing on the two known bifurcation failures (Euler buckling symmetry-breaking, critical-Reynolds branch selection) and measuring the false-positive rate on the 132 non-bifurcation cases is required before these can be deployed. We report them as a concrete proposal for closing the residual gap, not as a completed solution.

Adversarial test construction

50 inputs in three categories: (a) 20 ambiguous/incomplete (testing $\mathcal{A}_P$); (b) 20 with subtle well-posedness issues (testing $\mathcal{A}_J$); (c) 10 where L^2 correctness diverges from qualitative correctness (testing the quality audit). Full list in Supplementary Table S3.

Implementation and reproducibility

Python. LLM backbone: Claude-3.5-Sonnet (Anthropic, version 20241022), temperature 0. Primitives: NumPy, SciPy, PyTorch. All code, `spec.md` files, cached inference logs, the 72 prospective tasks with expert baselines, and a Docker container are archived on Zenodo (DOI: 10.5281/zenodo.XXXXXXX; to be minted before submission) and at https://github.com/integritynoble/Physics_World_Model. Exact reproduction commands in `REPRODUCE.md`. The Docker container pins all dependency versions and reproduces the 12 development problems without network access.

Independent replication. Three researchers (J. Rodriguez, U. Michigan; A. Patel, Georgia Tech; M. Chen, Stanford) independently ran the 12 development problems on separate hardware with no additional instructions. All 12 produced bounded-error solutions. Variability: ± 0.2 dB

237 (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic). Report in Supplementary Section S6.

238 **Data and code availability**

239 LoDoPaB-CT [12] (CC BY 4.0). Marmousi-2 [14] (public, SEG). GRI-Mech 3.0 [17] (public).
240 Source code, `spec.md` files, cached LLM logs, and Docker container under MIT license.

241 **Competing interests**

242 C.Y. is the founder of NextGen PlatformAI C Corp, which develops the platform described
243 here. Mitigation: (1) independent coordinator (Dr. M. Torres, ETH Zürich); (2) 12 external
244 scientists with no company relationship; (3) all code, data, and logs publicly archived; (4) three
245 independent researchers replicated results on separate hardware.

246 **AI-use disclosure**

247 The framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for code generation and
248 validation. The manuscript was drafted by the author with AI assistance for copy editing. All
249 scientific claims, experimental design, and interpretation are solely the author’s.

250 **Acknowledgements**

251 We thank the 12 benchmark scientists (Supplementary Table S2; names to be disclosed with
252 consent in the published version), the five blind-challenge participants, J. Rodriguez, A. Patel,
253 and M. Chen for replication, and Dr. M. Torres for independent coordination.

Table 2. Extended Data Table 1: Controlled comparison on 12 development problems. Five conditions isolate the Judge’s contribution. “Correct”: solution within domain-specific tolerance. “Bound”: automated error bound provided. Each condition run $3\times$ with different seeds; majority vote reported. Note: the 100% success rate on dev problems is expected, as these problems were used to design the Judge’s checks.

	Full pipeline		No $\mathcal{A}_J$		GPT-4+spec		GPT-4 raw		PINN
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT reconstruction	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ignition	✓	✓	~	—	~	—	—	—	—
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	~	—	~	—	~	—	—
BFS turbulent ^a	✓	✓	—	—	—	—	—	—	~
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat equation	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diffraction	✓	✓	✓	—	✓	—	✓	—	✓
Rosby waves	✓	✓	~	—	~	—	~	—	—
Total	11	11	7	0	6	0	5	0	4

✓ = correct within tolerance. ~ = partially correct or qualitatively wrong. — = incorrect or not provided. PINN = DeepXDE [19], PDE problems only. ^aJHU Turbulence Database [18]. COVID-19 SEIR moved to Supplementary Section S3 (model-class ceiling limits it to a case study).

Table 3. Extended Data Table 2: Comparison with existing tools. Each excels in its domain; the contribution here is automated validation of AI-generated code. Quantitative reference: on a standard Poisson problem ($\nabla^2 u = f$ on $[0, 1]^2$, 128^2 grid), framework error is 3.1×10^{-5} ; FEniCS achieves 2.8×10^{-5} ; both match the $O(h^2)$ theoretical rate.

	This work	COMSOL	FEniCS	OpenFOAM	PINNs	LLM agents
NL input	✓	—	—	—	—	✓
Cross-domain	Tested	Multi-phys.	PDE	CFD	PDE	Partial
Error bound	✓	—	A post.	—	—	—
Quality audit	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
Setup (median)	11 min	Hours	Hours	Hours	Hours	Minutes

A post. = a posteriori estimates for supported elements. LLM agents = Coscientist [6], ChemCrow [7].

255 **References**

- 256 [1] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440,
257 2021.
- 258 [2] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*,
259 2024.
- 260 [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J.*
261 *Comput. Phys.*, 378:686–707, 2019.
- 262 [4] A. Logg, K.-A. Mardal, and G. Wells (eds). *The FEniCS Book*. Springer, 2012.
- 263 [5] F. Rathgeber et al. Firedrake: automating the finite element method. *ACM TOMS*, 43(3):1–
264 27, 2016.

- [6] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [7] A. M. Bran et al. ChemCrow: augmenting LLMs with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.
- [8] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [9] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9:267–293, 1956.
- [10] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.
- [11] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.
- [12] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.
- [13] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.
- [14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25:156–166, 2006.
- [15] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60:1457–1473, 1995.
- [16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1:e8, 2013.
- [17] G. P. Smith et al. GRI-Mech 3.0. <http://combustion.berkeley.edu/gri-mech/>, 1999.
- [18] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.
- [19] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63:208–228, 2021.
- [20] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54:135–163, 2015.
- [21] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.